Project Report on

Ucab – Cab Booking System

(Full Stack Developer – MERN)

1. Project Title

UCab - Full-Stack Taxi Booking Application

2. Project Description

UCab is a full-stack taxi booking platform supporting two roles: riders and drivers. Riders can register, log in, select pickup/drop on a map, book and track rides, rate drivers, view history, make payments, and raise support tickets. Drivers can register/login, receive real-time ride requests, accept rides, and manage ride history.

Core Technology Used:

- Frontend: React (Vite), React Router, Axios, Bootstrap, Leaflet, Socket.IO client, SweetAlert2
- Backend: Node.js, Express, MongoDB, Mongoose, JWT, bcryptjs, Socket.IO
- Database: MongoDB (database: ucab)

Target Users:

- Riders (end users)
- Drivers (partners)
- (Optional) Admins

Real-world Relevance:

- Urban mobility and last-mile transport
- Demand-based pricing and supply balancing
- Low-latency dispatch using websockets

3. Application Scenarios / Use Cases

- Daily commute and point-to-point rides
- Airport and intercity drop services
- Corporate employee transport scheduling
- Driver partner job allocation and earnings tracking

4. Technical Architecture Overview

High-level Architecture:

- React SPA communicates with Express REST APIs
- JWT Bearer tokens for authentication/authorization
- MongoDB stores users, drivers, bookings, payments, support tickets
- Socket.IO channels for real-time ride request and acceptance

(Diagram placeholder) Architecture and request/data flow diagram to be inserted.

5. Prerequisites

5.1 Software Requirements

- Node.js 18+ and npm/yarn
- MongoDB Community Server or Atlas
- Vite (bundler) and a modern browser
- IDE: VS Code

5.2 Libraries / Frameworks / Dependencies

- Frontend: React (Vite), React Router, Axios, Bootstrap, Leaflet, Socket.IO client, SweetAlert2
- Backend: Node.js, Express, MongoDB, Mongoose, JWT, bcryptjs, Socket.IO

5.3 Hardware Requirements (If Applicable)

- Developer machine with 8GB RAM+
- GPS-enabled device for field testing (optional)

6. Prior Knowledge Required

- JavaScript/TypeScript basics
- React fundamentals (state, hooks, routing)
- Node.js + Express APIs
- MongoDB + Mongoose ODM
- JWT authentication and authorization
- Socket.IO event-driven programming
- Leaflet map basics

7. Project Objectives

- Deliver a working MVP enabling rider-driver real-time interaction
- Maintain secure authentication and protect PII
- Ensure responsive UI and low latency for socket events
- Prepare codebase for future scale and admin workflows

8. System Workflow

Rider Workflow:

1. Register/Login
2. Select pickup & drop on map and book

3. Wait for driver acceptance (real-time)
4. Track ride status and route
5. Rate driver and optionally make payment

Driver Workflow:

6. Register/Login as driver
7. Join vehicle-type room and receive requests
8. Accept one ride request
9. Mark ride completed and view history

9. Milestone 1: Requirement Analysis & System Design

Problem Definition:

Enable quick, reliable cab booking and real-time driver allocation with transparent status tracking and post-ride workflows.

Functional Requirements:

- User registration/login with role (rider, driver)
- Password hashing with bcrypt
- Ride booking with pickup/drop coordinates and fare
- Driver dashboard for pending requests and ride completion
- Ride tracking page with simulated movement and status phases
- Rider rating system for completed rides
- Payment creation and payment history
- Support ticket creation and listing

Non-Functional Requirements:

- Performance: p95 < 500ms for common API calls; socket latency < 1s (local)
- Security: JWT, bcrypt hashing; restrict CORS; secrets backend-only
- Scalability: stateless APIs, socket rooms per vehicle type
- Logging/Monitoring: request logs and centralized error handling

System Design Decisions:

- MERN stack for rapid full-stack development
- JWT for stateless auth; role-based middleware
- Socket.IO for bi-directional events
- Leaflet for open-source mapping

Technology Stack Selection Justification:

- React + Vite: fast DX and SPA routing

- Express + Mongoose: lightweight API with flexible schema
- MongoDB: document model suits dynamic booking payloads

(Diagrams placeholder) Include Architecture Diagram and ER Diagram here.

10. Milestone 2: Environment Setup & Initial Configuration

10.1 Development Environment Setup

- Install Node.js and MongoDB
- Clone repo; set up backend and frontend folders

10.2 Dependency Installation

- Backend: npm install (express, mongoose, cors, jsonwebtoken, bcryptjs, socket.io)
- Frontend: npm install (react, react-router-dom, axios, bootstrap, leaflet, socket.io-client, sweetalert2)

10.3 Project Structure Creation

See Section 15 for detailed structure.

10.4 Configuration Setup

- Backend .env: MONGO_URI, JWT_SECRET, PORT
- Frontend .env: VITE_API_BASE_URL (avoid secrets on FE)

11. Milestone 3: Core System Development

11.x Feature – Authentication & Roles

- Description
- Key API endpoints
- UI screenshots (to be inserted)

11.x Feature – Booking Creation & Retrieval

- Description
- Key API endpoints
- UI screenshots (to be inserted)

11.x Feature – Real-time Matching via Socket.IO

- Description
- Key API endpoints
- UI screenshots (to be inserted)

11.x Feature – Ride Tracking & Completion

- Description

- Key API endpoints
- UI screenshots (to be inserted)

11.x Feature – Rating & Feedback

- Description
- Key API endpoints
- UI screenshots (to be inserted)

11.x Feature – Payments & History

- Description
- Key API endpoints
- UI screenshots (to be inserted)

11.x Feature – Support Tickets

- Description
- Key API endpoints
- UI screenshots (to be inserted)

12. Milestone 4: Integration & Optimization

- Component integration between auth, booking, and realtime modules
- Performance optimization and request caching where feasible
- Security hardening (rate limiting, input validation)
- Centralized error handling and user-friendly alerts

13. Milestone 5: Testing & Validation

13.1 Test Cases (Sample)

Test Case ID	Input	Expected Output	Status
TC-001	POST /api/auth/register with valid payload	201 Created, JWT token returned	Pass/Fail
TC-002	POST /api/bookings with valid token	200 OK, booking created with status=pending	Pass/Fail
TC-003	Socket event joinDriverRoom	Driver receives newRideRequest when booking created	Pass/Fail

13.2 Unit Testing • 13.3 Integration Testing • 13.4 UAT • 13.5 Performance Metrics

Planned: Jest/Mocha for backend, React Testing Library for frontend; supertest for API; Lighthouse for performance.

14. Deployment

- Deployment Architecture: Backend (Node) + MongoDB + Frontend (static hosting)
- Hosting: Any Node-compatible PaaS or container platform
- Steps: Build FE → Deploy static; Deploy BE with env vars; Configure CORS & HTTPS
- Production Considerations: logging, backups, monitoring, domain & TLS

15. Project Structure

backend/src/models: User, Driver, Booking, Payment, SupportTicket

backend/src/routes: auth, bookings, drivers, payments, support

backend/src/middleware: JWT & role middleware

backend/src/server.js: app entry, DB connection, socket setup

frontend/src/pages: UI pages for rider/driver flows

frontend/src/App.jsx: route configuration

16. Results

- End-to-end MVP working for rider-driver booking lifecycle
- Realtime acceptance via Socket.IO
- Booking, payments, and support records persisted in MongoDB

17. Advantages & Limitations

- Advantages: Real-time architecture; clear modular separation; open-source mapping
- Limitations: Endpoint/field mismatches; lack of admin role; open CORS; no automated tests

18. Future Enhancements

- Implement missing endpoints or remove unsupported frontend pages
- Add admin role and secure admin workflows
- Add paymentStatus to booking schema or adjust logic
- Move API base URL to frontend env (VITE_API_BASE_URL)

- Add validation (Joi/Zod), logging, centralized error handling
- Add unit/integration tests and API documentation (Swagger/OpenAPI)

19. Conclusion

UCab demonstrates a strong end-to-end MVP for a taxi booking platform, especially in real-time rider–driver interaction. Next steps include aligning frontend–backend contracts, adding security hardening, and increasing test coverage.

20. Appendix

- Source code and configuration files
- API documentation (Swagger/OpenAPI) – planned
- GitHub/Repo link – https://github.com/Pravin-junne/Ucab-
- Live demo link – https://sbupune-my.sharepoint.com/personal/pravin_mca2504034_sbup_edu_in/_layouts/15/stream.aspx?id=%2Fpersonal%2Fpravin%5Fmca2504034%5Fsbup%5Fedu%5Fin%2FDocuments%2Fdemo%20video%2FScreen%20Recording%202026%2D03%2D09%20173727%2Emp4&referrer=StreamWebApp%2EWeb&referrerScenario=AddressBarCopied%2Eview%2Ef7174183%2D1a60%2D45ad%2D8e36%2D93f429b2ffae